

Project Documentation

Student Name: Rosine Kitho

Admission Number: BIT/2025/33559

Unit Code: BIT4107

Institution: Mount Kenya University

Supervisor: Mr. Nyoro

WEEK 1: INTRODUCTION TO MOBILE DEVELOPMENT

A. Install all development tools

Tools Used

1. Flutter

I developed the mobile application using Flutter.

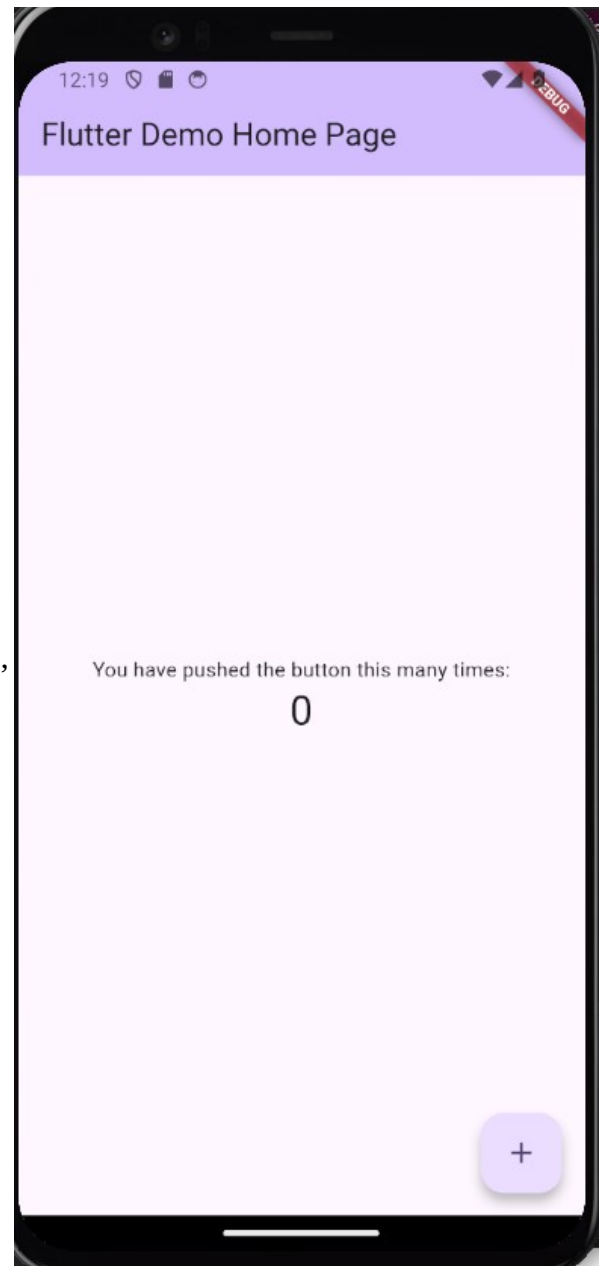
Flutter is a framework that allowed me to create the mobile application's modern user interface and create applications on multiple platforms using one codebase.

1.1. Installing the Flutter SDK

To install Flutter on my computer, I downloaded the SDK from the official Flutter website, unzipped it and added it to my PATH.

After installing Flutter, I verified that it was installed correctly by running the command:

<flutter doctor>



2. Dart

Dart is the programming language used by Flutter.

2.1. Installation

Dart was installed automatically as part of the Flutter SDK.

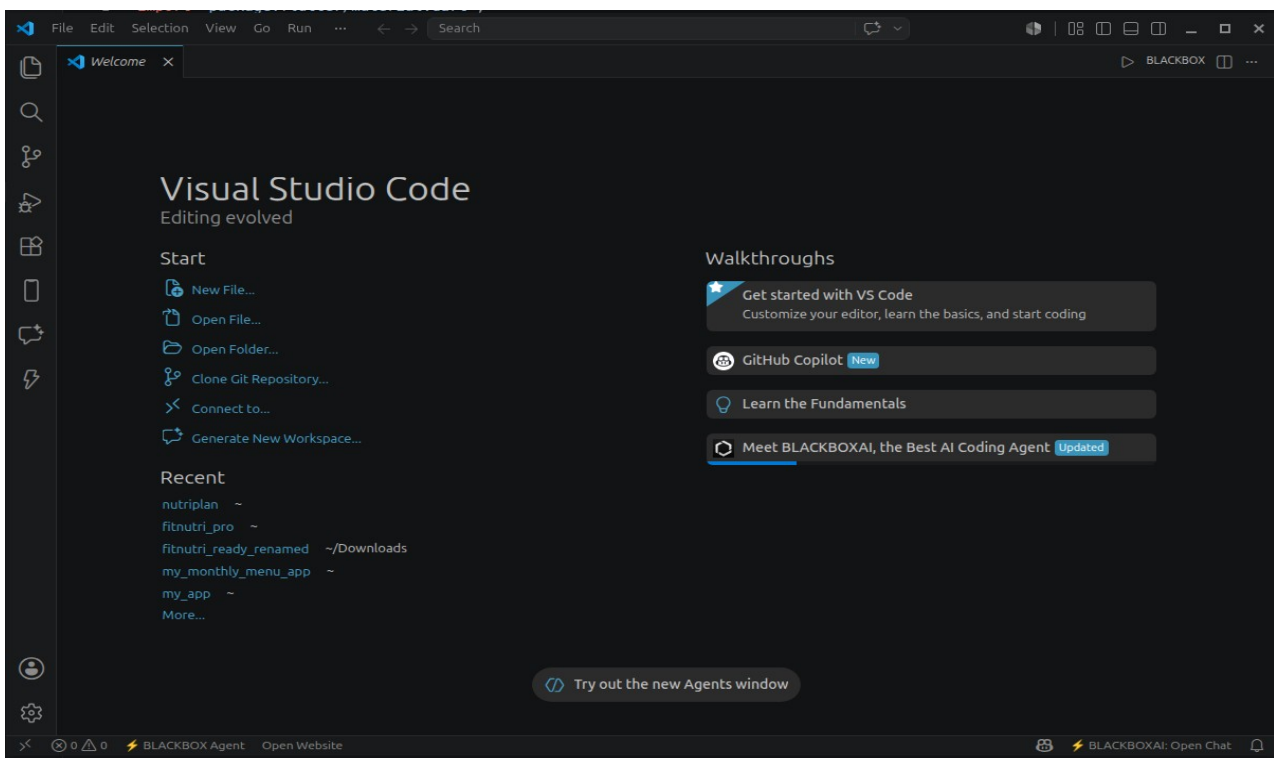
```
roxae@roxae-ThinkPad-T490s:~/nutriplan$ dart --version
Dart SDK version: 3.12.1 (stable) (Tue May 26 01:02:21 2026 -0700) on "linux_x64"
roxae@roxae-ThinkPad-T490s:~/nutriplan$
```

3. Visual Studio Code

Visual Studio Code was my main code editor.

3.1. Installation

I downloaded and installed Visual Studio Code and added the Flutter and Dart extensions.

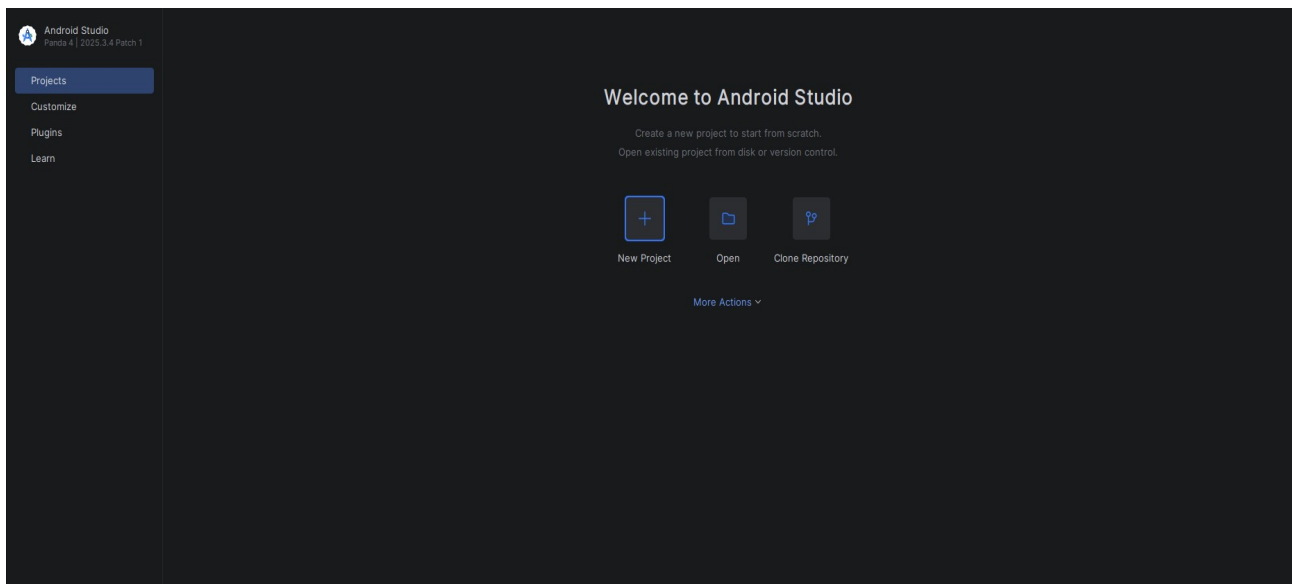


4. Android Studio

Android Studio was used to install the Android SDK and manage Android emulators.

4.1. Installation

I downloaded Android Studio, installed the Android SDK, and created an Android Virtual Device (AVD) for testing.



5. Git and GitHub

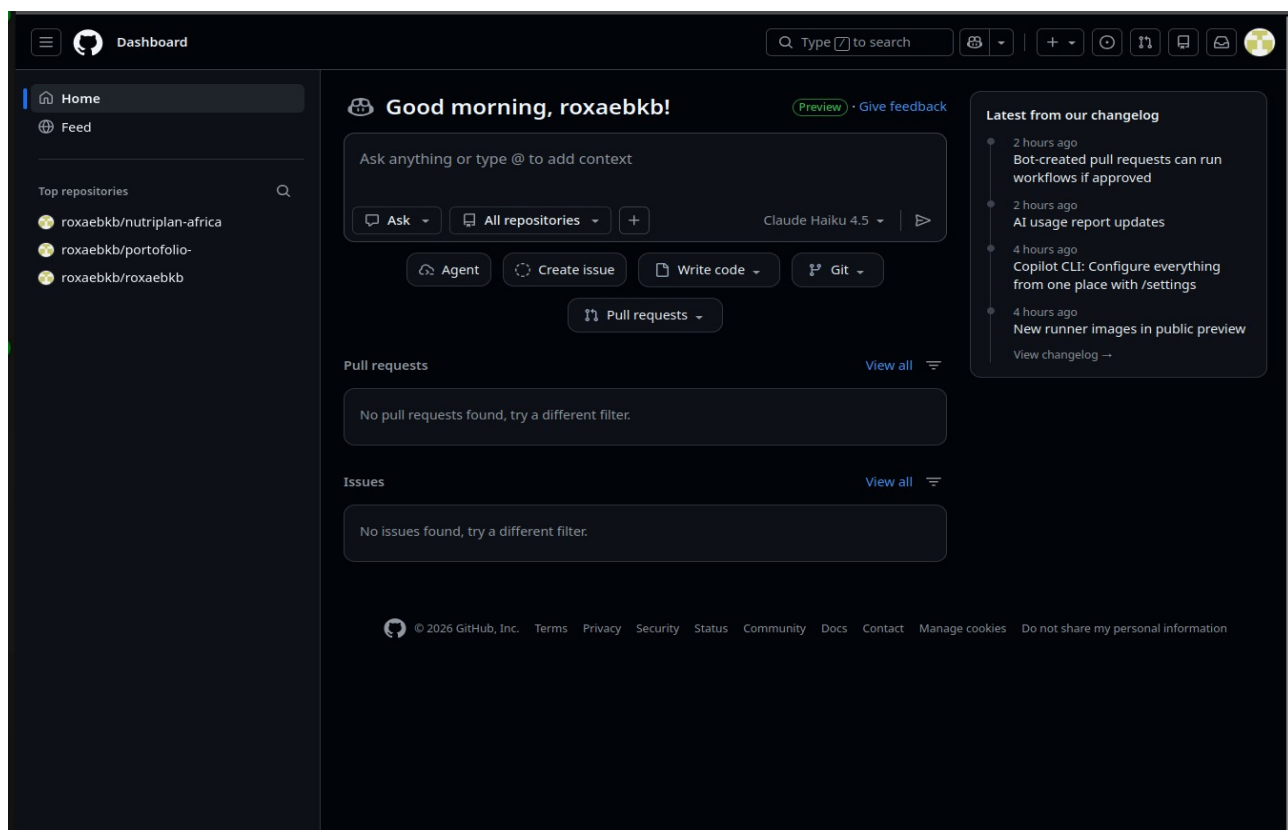
Git and GitHub were used to manage the source code and keep track of project changes.

5.1. Installation

Git was installed using:

```
sudo apt install git
```

```
roxae@roxae-ThinkPad-T490s:~$ git --version
git version 2.43.0
roxae@roxae-ThinkPad-T490s:~$
```



B. Creation of one Flutter app



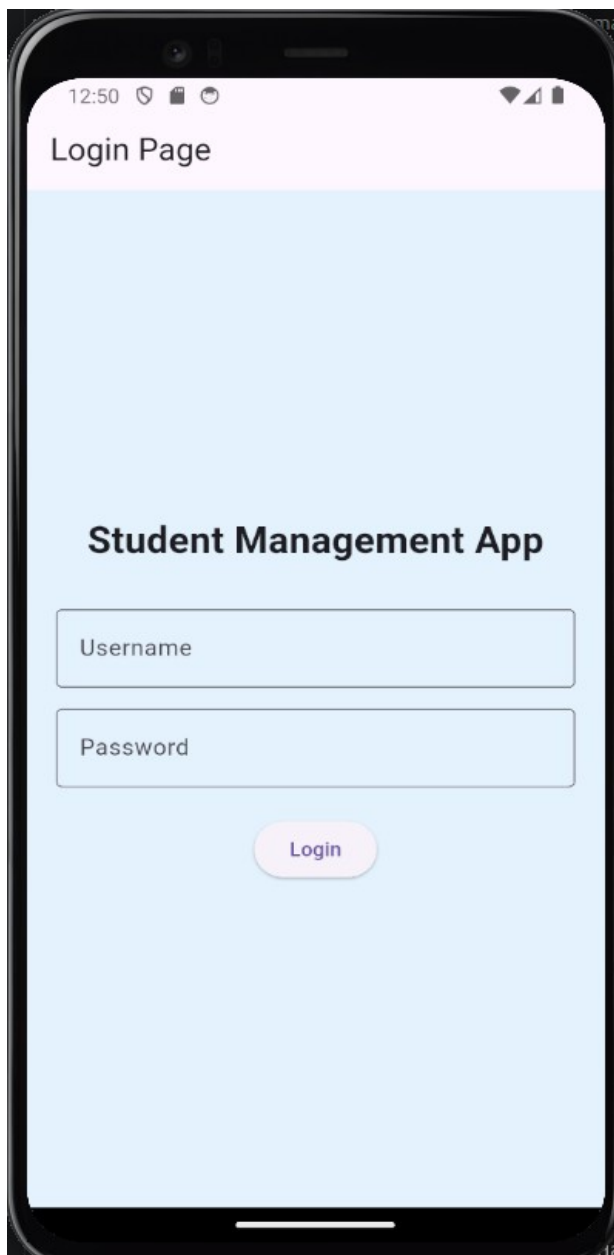
```
lib > main.dart > MyApp
1  import 'package:flutter/material.dart';
2
   Run | Debug | Profile
3  void main() {
4    | runApp(const MyApp());
5  }
6
7  class MyApp extends StatelessWidget {
8    const MyApp({super.key});
9
10   @override
11   Widget build(BuildContext context) {
12     return MaterialApp(
13       title: 'My First Flutter App',
14       debugShowCheckedModeBanner: false,
15       home: Scaffold(
16         backgroundColor: Colors.orange,
17         appBar: AppBar(
18           title: const Text(
19             'My First Flutter App',
20             style: TextStyle(fontSize: 24),
21           ), // Text
22           backgroundColor: Colors.blue,
23         ), // AppBar
24         body: const Center(
25           child: Text(
26             'Hello Flutter!',
27             style: TextStyle(
28               fontSize: 40,
29               fontWeight: FontWeight.bold,
30             ), // TextStyle
31           ), // Text
32         ), // Center
33       ), // Scaffold
34     ); // MaterialApp
35   }
36 }
```

WEEK 2: PROGRAMMING LANGUAGES AND FRAMEWORKS

Mini Student Management App

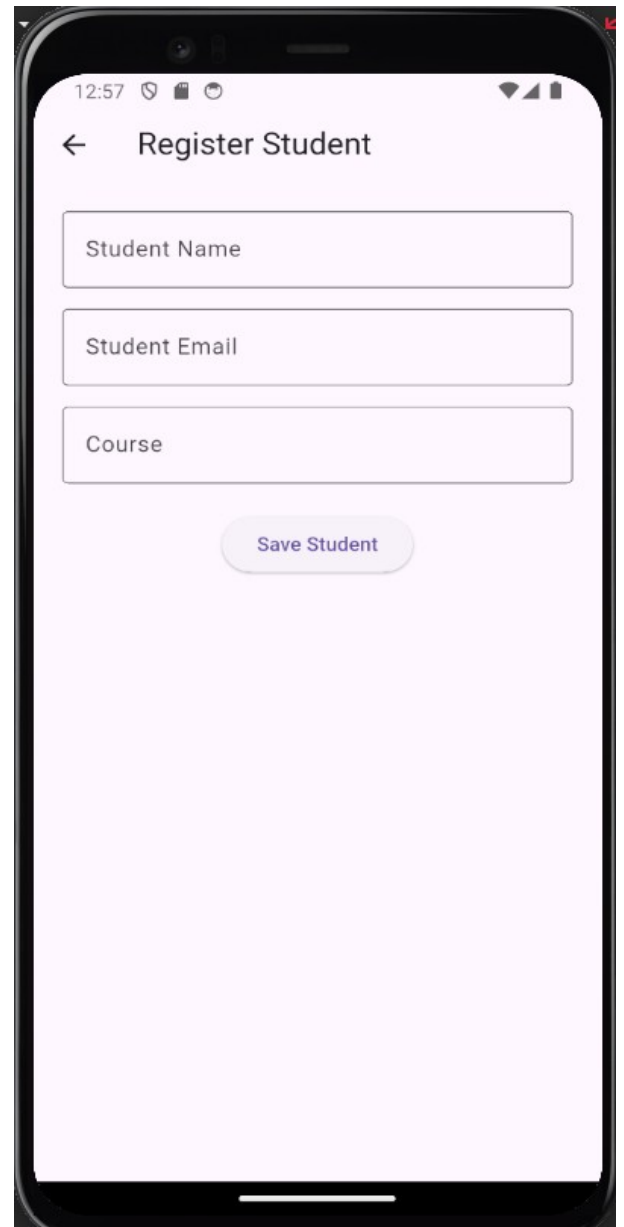
Features:

Login page



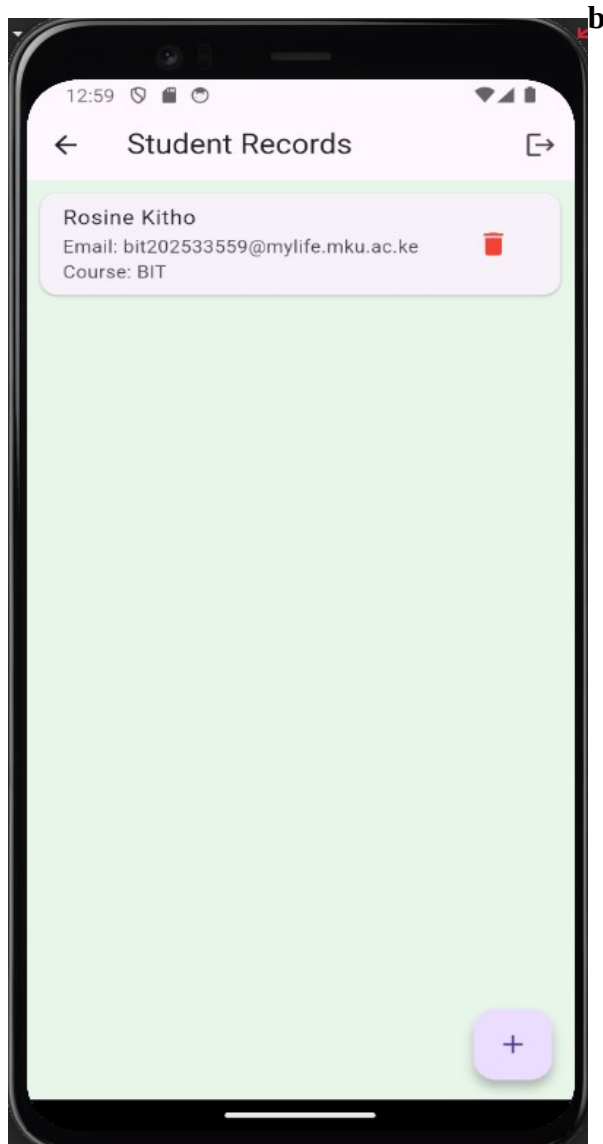
A mobile app login page with a light blue background. At the top, the status bar shows the time 12:50 and various icons. Below the status bar, the text "Login Page" is displayed in a light pink header. The main content area features the title "Student Management App" in bold black text. Below the title, there are two input fields: "Username" and "Password", both with light blue borders. At the bottom, there is a pink "Login" button with rounded corners.

Student registration

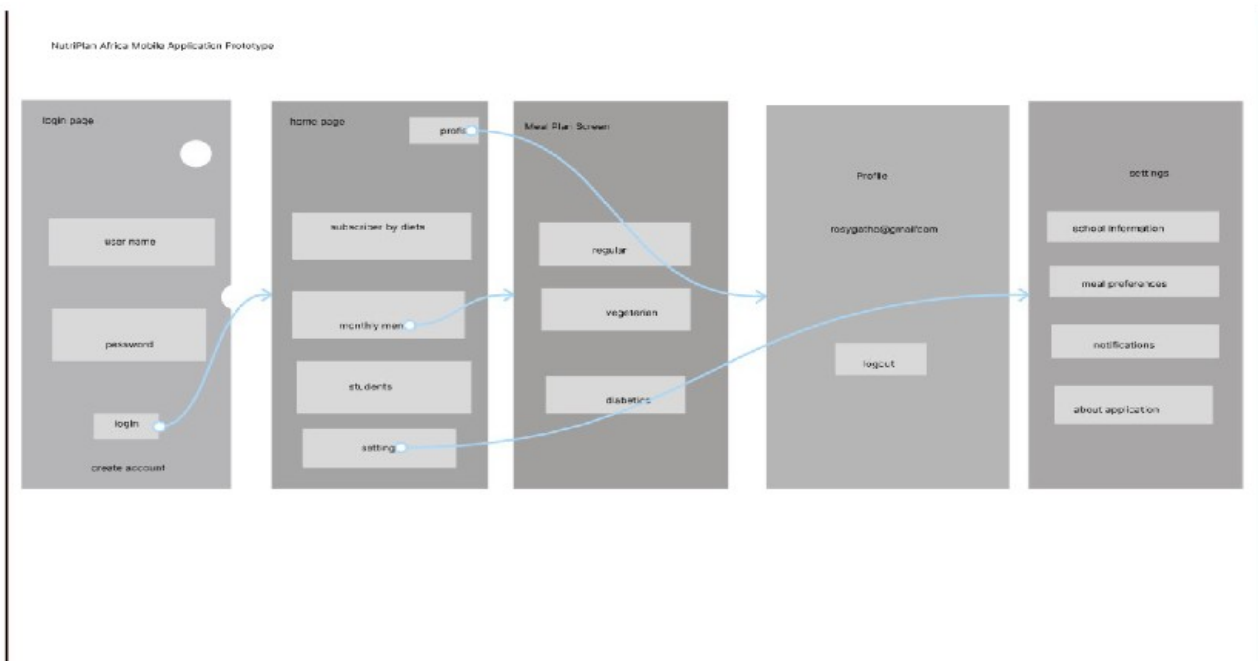


A mobile app student registration page with a light pink background. At the top, the status bar shows the time 12:57 and various icons. Below the status bar, the text "Register Student" is displayed in a light pink header, preceded by a back arrow. The main content area features three input fields: "Student Name", "Student Email", and "Course", all with light pink borders. Below these fields, there is a pink "Save Student" button with rounded corners.

Save records locally



WEEK 3: USER INTERFACE (UI) DEVELOPMENT [UI Prototype]



My mobile application

About NutriPlan Africa

NutriPlan Africa is a mobile application designed to help users improve their nutrition and maintain a healthy lifestyle. The application provides meal plans based on different dietary needs, including regular, vegetarian, and diabetic diets.

Users can browse meal recommendations, manage their preferences, and access nutrition-related information through a simple and user-friendly interface.

The goal of NutriPlan Africa is to make healthy eating more accessible and help individuals make informed food choices for better health and well-being.

Problem Statement

Many schools face challenges such as:

- Poor management of student nutrition records.
- Difficulty tracking dietary requirements.
- Lack of centralized student information.
- Time-consuming manual record keeping.
- Limited reporting capabilities.

These challenges reduce efficiency and increase the possibility of errors.

Proposed Solution

NutriPlan Campus provides a mobile application that enables institutions to:

- Register and manage students.
- Manage dietary categories.
- Store records securely in the cloud.
- Generate nutrition reports.
- Improve nutrition management efficiency.

System Features

Figure 1: Login Screen

This screen allows registered users to securely access the application using their email address and password. Authentication is handled through Firebase Authentication.

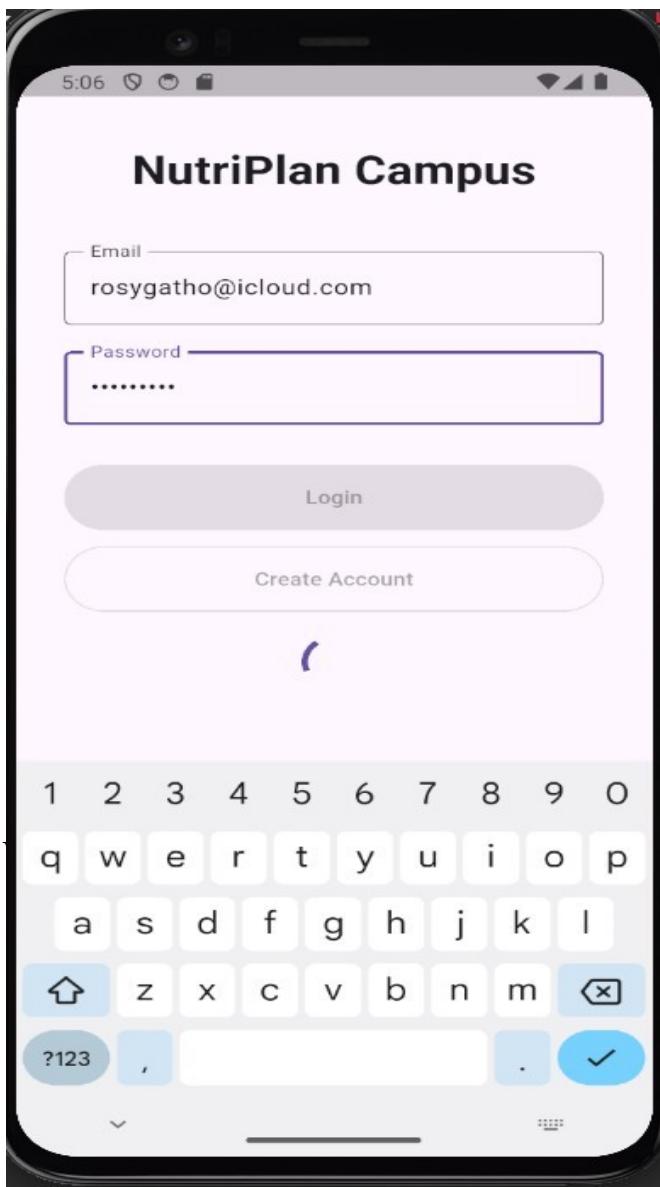
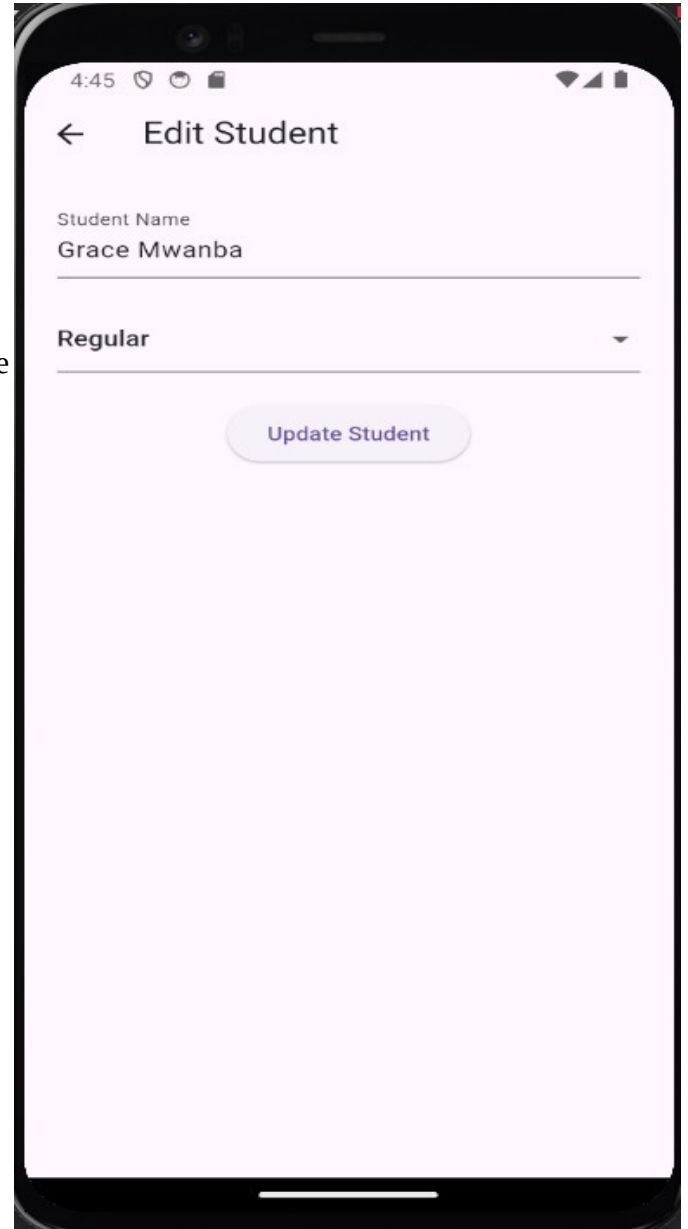


Figure 2: Create Account Screen

This screen enables new users to create an account by providing their email address and password. The account information is securely stored using Firebase Authentication.

Figure 3: Home Dashboard

The dashboard serves as the main navigation area of the application. It provides quick access to student management, meal planning, reports, and other system features.

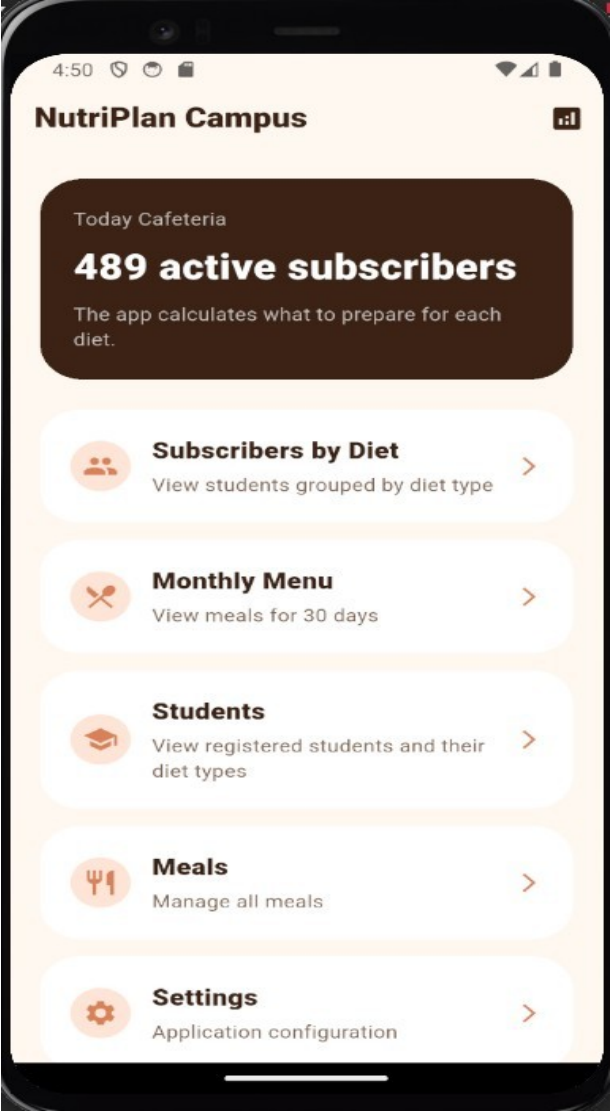
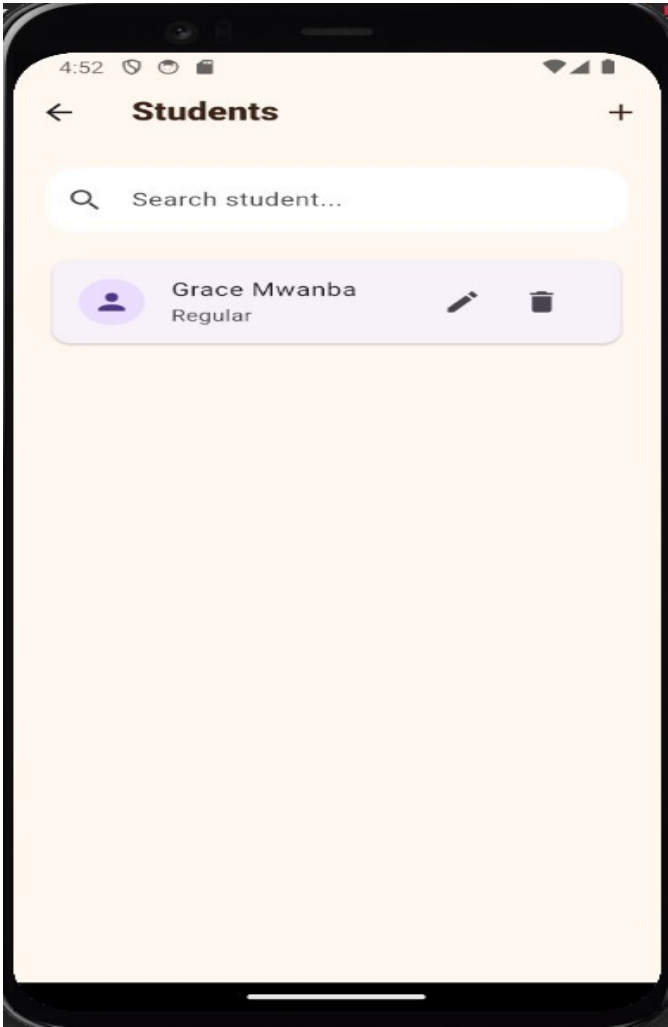


Figure 4: Students Screen

This screen displays all registered students. Users can search, view, edit, and delete student records from the database.



Z

Figure 5: Add Student Screen

This screen allows administrators to register new students and assign the appropriate dietary category.

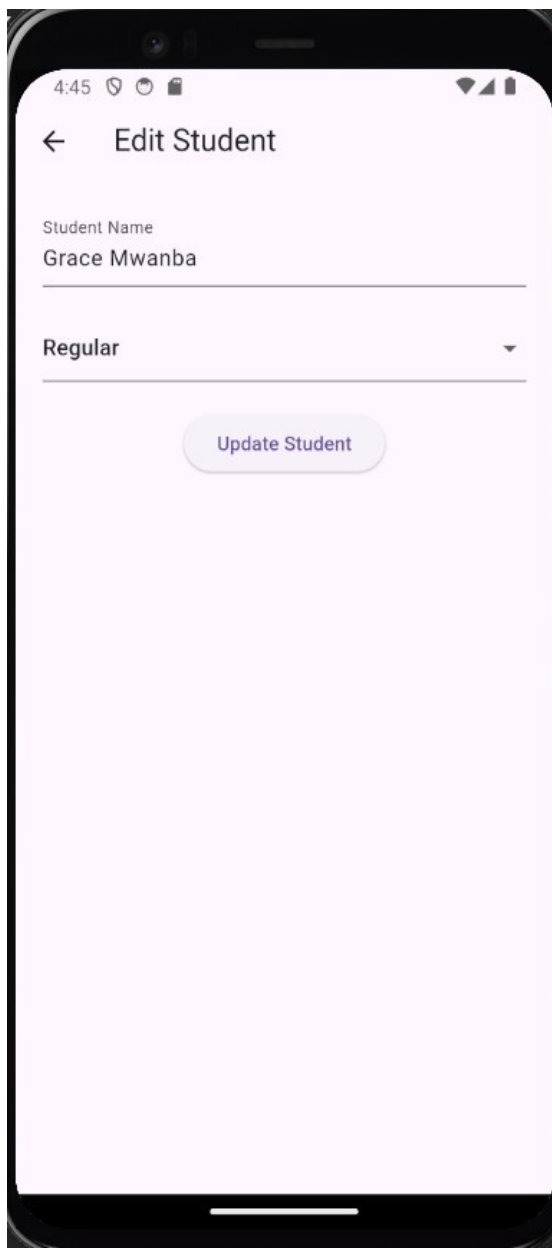
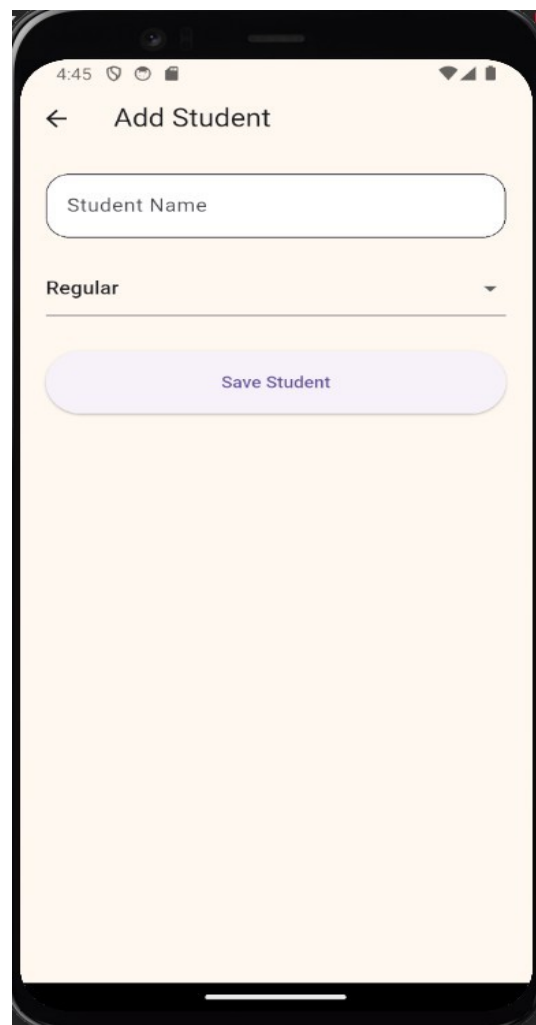


Figure 6: Edit Student Screen

This screen enables users to update student information and modify dietary requirements when necessary.

Figure 7: Reports Screen

The reports section provides statistical information about students and dietary categories, helping administrators make informed decisions.

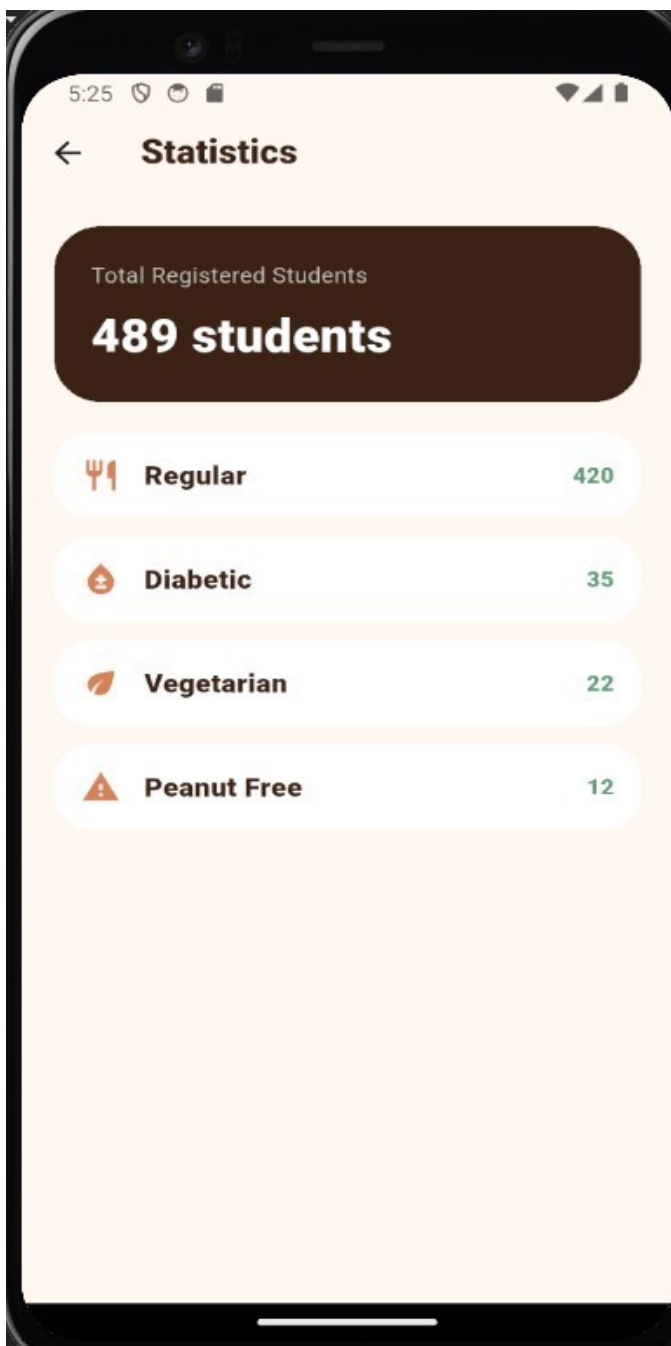
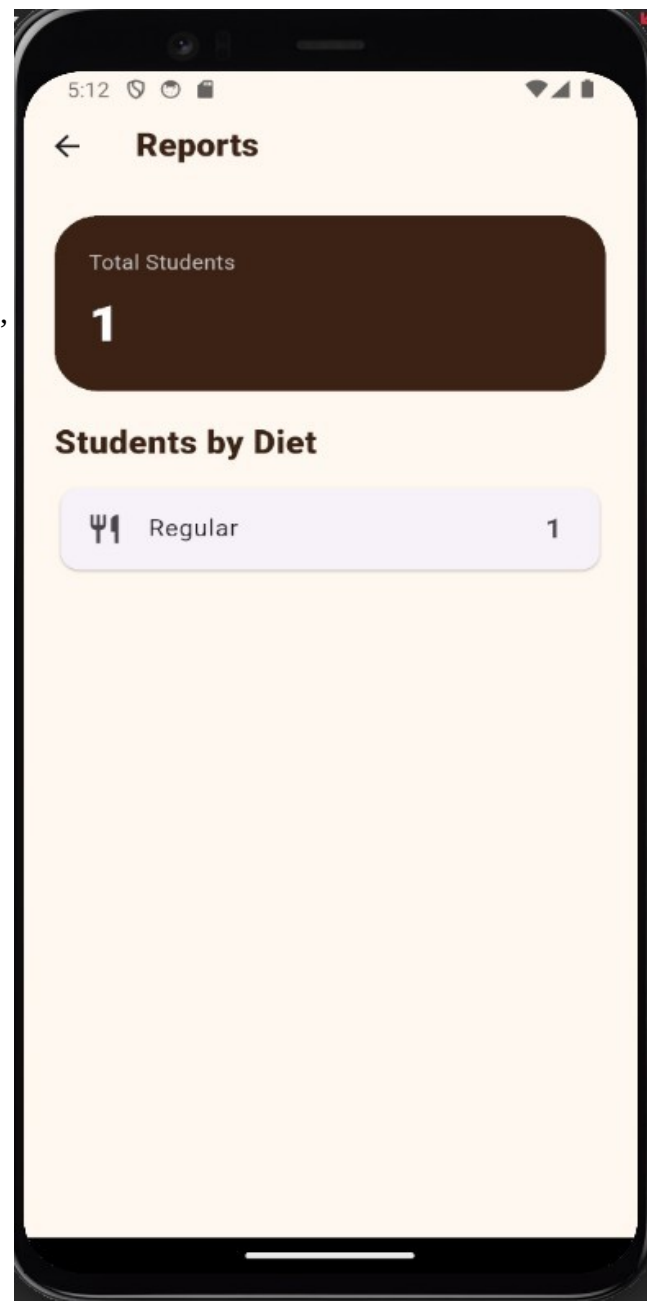


Figure 7: Statistics Screen

This screen displays statistical information about registered students. It shows the total number of students and the number of students assigned to each dietary category. This information assists administrators in planning meals according to student dietary needs.

Figure 9: Monthly Menu Screen

This screen provides access to the cafeteria's monthly meal plan. Each day is represented by a menu card that allows users to view the meals scheduled for that date. The monthly menu helps ensure proper meal organization and supports dietary planning for different student categories.

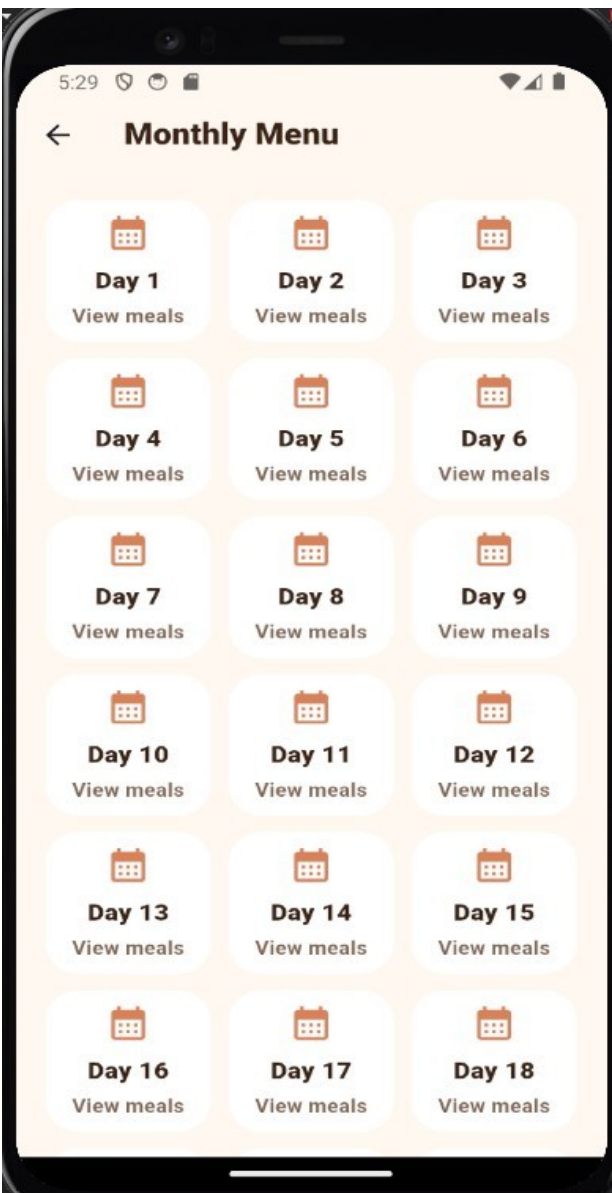


Figure 10: Cloud Firestore Database

This screenshot shows the students collection stored in Cloud Firestore. The database stores student information including the student's name, dietary category, and record creation date. Cloud Firestore provides real-time cloud storage and synchronization for the application.

